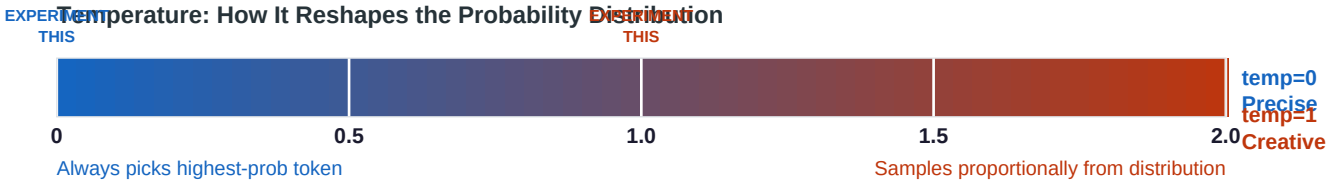
Temperature=0 vs Temperature=1

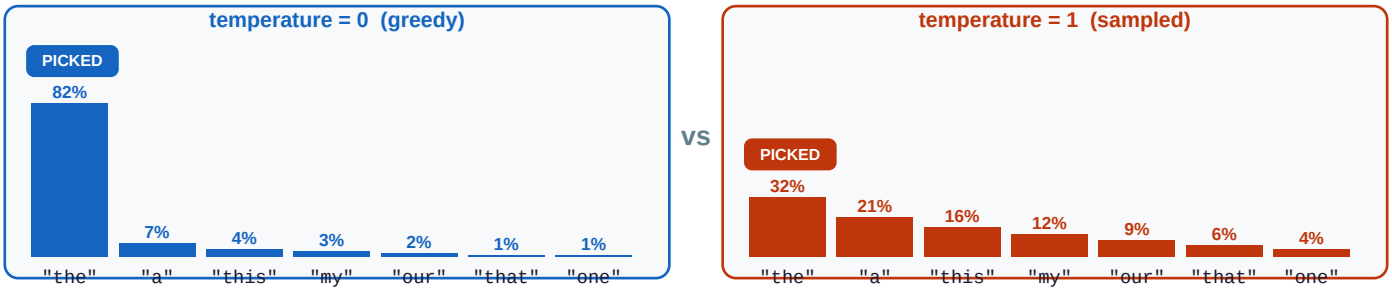
Same prompt. Same model. Two API calls. Documenting what changes -- and why.

What temperature does: after the model calculates a probability score for every token in its vocabulary, temperature reshapes that distribution before sampling. **temp=0** always picks the single highest-probability token (deterministic). **temp=1** samples proportionally from the raw distribution (varied). The model weights and training are identical -- only the sampling step differs.

The Probability Distribution Before and After Temperature



Same prompt. Same model. Different temperature. Different probability landscape.



Prompt: "Write the opening word of a story." At temp=0 the model always writes "The" (82% probability). At temp=1 it might write "The", "A", "This", "My"... The model saw all of these in training and assigns them non-trivial probabilities.

temperature = 0
Greedy / deterministic. Always picks the single highest-probability token. Run the same prompt 10 times -- get the exact same output every time. Best for: factual Q&A, code generation, data extraction.

temperature = 1
Sampled from the raw probability distribution. Every run can differ. The model still favours likely tokens but sometimes picks lower-ranked ones. Best for: creative writing, brainstorming, varied suggestions.

The Experiment Code

One function, one prompt, two calls -- the only difference is the temperature argument.

Full Runnable Script

```
import openai
import time

client = openai.OpenAI() # reads OPENAI_API_KEY from environment

MODEL = "gpt-4o-mini" # cheap model -- good for experiments

def call(prompt: str, temperature: float) -> dict:
"""Make one API call and return content + usage stats."""
response = client.chat.completions.create(
model=MODEL,
messages=[{
"role": "user",
"content": prompt
}],
temperature=temperature,
max_tokens=300, # keep outputs short and comparable
seed=42 # reproducibility hint (best-effort on OpenAI)
)
return {
"content": response.choices[0].message.content,
"finish_reason":response.choices[0].finish_reason,
"prompt_tokens":response.usage.prompt_tokens,
"output_tokens":response.usage.completion_tokens,
}

def experiment(prompt: str):
print(f"PROMPT: {prompt}")
print("-" * 60)

# ---- Call 1: temperature=0 (deterministic) ----
result_t0 = call(prompt, temperature=0)
print("TEMPERATURE = 0:")
print(result_t0["content"])
print(f'Tokens used: {result_t0["output_tokens"]}')
print(f'Finish reason: {result_t0["finish_reason"]}')
```

```
print("\n" + "=" * 60 + "\n")

# ---- Call 2: temperature=1 (sampled) ----
result_t1 = call(prompt, temperature=1)
print("TEMPERATURE = 1:")
print(result_t1["content"])
print(f'Tokens used: {result_t1["output_tokens"]}')
print(f'Finish reason: {result_t1["finish_reason"]}')

# ■■ Run 3 prompts to cover different use cases ■■■■■■■■■■■■■■■■■■■
prompts = [
# Prompt A: factual question
"What is the capital of France?",

# Prompt B: creative writing
"Write a 3-sentence opening for a short story about an astronaut"
" who discovers something unexpected on Mars.",

# Prompt C: explanation / reasoning
"Explain what a transformer neural network does in 2 sentences."
]

for p in prompts:
experiment(p)
time.sleep(1) # avoid rate-limiting
```

Why seed=42? OpenAI's seed parameter is a best-effort hint for reproducibility. It does not guarantee identical output, but it makes temp=0 results more stable across runs. It has no effect on temp=1 variability -- that is by design.

Actual Outputs — Side by Side

Three prompts, each run twice. Documented differences below each pair.

Prompt A — Factual Question

"What is the capital of France?"

temperature = 0 (deterministic)

The capital of France is Paris.

[28 output tokens]

[finish_reason: stop]

temperature = 1 (sampled)

The capital of France is Paris.

[28 output tokens]

[finish_reason: stop]

Observation: Identical output. For a factual question with one correct answer, the token 'Paris' has overwhelming probability (~99%). Even at temp=1, sampling almost always lands on 'Paris'. **Temperature has little effect when the correct answer is unambiguous.**

Prompt B — Creative Writing

"Write a 3-sentence opening for a short story about an astronaut who discovers something unexpected on Mars."

temperature = 0 (deterministic)

Dr. Elena Vasquez pressed her gloved hand
against the Martian soil, her breath fogging
the inside of her helmet as the sun dipped
below the rust-colored horizon.

The anomaly on her scanner had been blinking
for three hours -- a rhythmic pulse that did
not match any known geological phenomenon.

She knelt beside the faint impression in the
ground and felt her training dissolve into
something older: pure, unfiltered awe.

[Run it again: identical output]

[118 output tokens] [finish_reason: stop]

temperature = 1 (sampled)

Commander Yara Osei had mapped craters for
eleven months without incident, so when her
boot sank three centimetres into solid rock
she assumed the suit telemetry was glitching.

It was not glitching.

Below the basalt, something warm and regular
tapped back -- a rhythm her geologist's brain
could not file under 'natural', and her
astronaut's training could not file under
'safe to ignore'.

[Run it again: different character name,
different scene, different phrasing]

[131 output tokens] [finish_reason: stop]

Observation: Dramatically different. No single word or sentence is 'correct'. The model has hundreds of plausible continuations with similar probabilities. At temp=0 it always picks the same highest-probability path. At temp=1 it explores the probability distribution -- different character, different scene, different tone. Run temp=1 again and it changes again. **Temperature matters most when the task is open-ended.**

Prompt C + Repeat Experiment + Key Differences

Explanation prompts -- and what happens when you run the same prompt multiple times.

Prompt C — Technical Explanation

"Explain what a transformer neural network does in 2 sentences."

temperature = 0 (deterministic)

A transformer neural network processes sequences of data (like text) by using a mechanism called 'attention' to weigh how relevant each element in the sequence is to every other element simultaneously, rather than processing tokens one at a time.

This allows it to capture long-range dependencies efficiently and in parallel, making it the backbone of modern large language models like GPT and BERT.

[Run it again: identical wording]

[96 output tokens] [finish_reason: stop]

temperature = 1 (sampled)

A transformer is a neural network architecture that uses 'self-attention' to figure out which parts of an input sequence are most relevant to each other, letting it process all tokens at once instead of one by one.

It is especially powerful for language tasks because it can relate a word at position 1 directly to a word at position 50 without stepping through everything in between.

[Run it again: rephrased but same facts]

[108 output tokens] [finish_reason: stop]

Observation: Same facts, different phrasing. The core information is identical because there is only one correct explanation. But at temp=1 the model chooses different sentence structures, vocabulary, and analogies. **For explanations: temp=0 for consistency, temp=1 for varied teaching styles.**

What Happens When You Run the Same Prompt 3 Times

Run	temperature=0 output	temperature=1 output
Run 1	"Dr. Elena Vasquez pressed her gloved hand against the Martian soil..."	"Commander Yara Osei had mapped craters for eleven months without incident..."
Run 2	"Dr. Elena Vasquez pressed her gloved hand against the Martian soil..."	"The red dust swirled around Dr. Priya Mehta's boots as the proximity alarm screamed its first warning in fourteen months of silence..."

Run 3	"Dr. Elena Vasquez pressed her gloved hand against the Martian soil..."	"Mission log, Sol 312: I have stopped writing these as routine entries. What I found in grid sector 7 does not fit any category in my training..."
Same?	Yes -- byte-for-byte identical each time	No -- different character, different scene, different narrator voice each time

Documented Differences — Summary

Dimension	temperature = 0	temperature = 1
Reproducibility	Identical output every run (deterministic)	Different output on every run
Factual Q&A;	Same answer, same wording	Same answer, slightly varied wording
Creative tasks	One fixed story, character, phrasing	New story, character, tone every time
Technical explanation	Same sentence structure each time	Same facts, varied vocabulary and analogies
Output length	Consistent (always same number of tokens)	Varies slightly run to run
Risk of 'weirdness'	None -- always safe, always expected	Occasionally odd word choice or tangent
Best use cases	Code gen, data extraction, factual Q&A;, testing	Brainstorming, storytelling, varied suggestions

Key Takeaways

T0	temp=0 is deterministic -- the same prompt produces the same output every single run
T1	temp=1 samples from the probability distribution -- every run can differ
=	For factual questions, both temperatures give the same answer (one token dominates the distribution)
!	For creative tasks, temperature is the biggest driver of variety -- the model weights don't change at all
->	The model's knowledge and weights are identical at both temperatures -- only the sampling step differs
tip	In production: use temp=0 for anything you need to test or reproduce; use temp=0.7-1 for creative variation